

Advantages of OOP & the Modern Java Landscape

Revised focus: Java platform, toolchain, bytecode, JVM execution, portability & developer workflow

CSC241 — Object-Oriented Programming
Fall 2026 • COMSATS University Islamabad, Lahore Campus
Dr. Muhammad Shahid Bhatti • Assistant Professor • Office H-17

Lecture 1 already established the motivation for OOP. Today we move quickly into the Java technology landscape.



Course Repository: github.com/mshahidbhatti/CSC241-OOP

Where Lecture 1 ended — and where Lecture 2 begins

We will not re-teach programming paradigms, procedural-vs-OOP motivation, or class/object previews.

Lecture 1 already established

- Programming paradigms and procedural organization
- Why large procedural systems become difficult to evolve
- Objects combine state, behavior, and identity
- OOP helps localize responsibilities and collaboration
- A first preview of classes, objects, and encapsulation

Lecture 2 adds the technology layer

- How Java source becomes executable behavior
- Java language vs Java SE vs JVM vs JDK
- Bytecode, class loading, interpreter and JIT
- Why the same class file can run on multiple platforms
- How IDEs, Git, and the JDK fit into your workflow

Think of today as: “Now that we know why OOP exists, what actually happens when we write and run Java?”

Lecture roadmap

A brief OOP bridge, then a deep dive into the modern Java execution model.



Main question for the lecture: “What does the Java platform do between writing source code and seeing output?”

Learning outcomes

OUTCOMES

By the end of this lecture, you should be able to...

Differentiate

Java language, Java SE, JVM, JRE concept, and JDK.

Trace

a program from .java source to .class bytecode to JVM execution.

Explain

why bytecode supports portability across operating systems and CPU architectures.

Use

core commands: java, javac, javap, jshell, and jar at a conceptual level.

Diagnose

the difference between compile-time failure and runtime failure.

Choose

a sensible semester JDK baseline and verify the active toolchain.

OOP benefits — one-slide carry-forward

Lecture 1 covered the motivation. Keep only these design consequences in mind as we move into Java.

Reuse

use stable abstractions
again

Maintain

change one clear owner

Scale

divide responsibilities

Test

check focused components

Extend

add behavior behind
contracts

Key transition

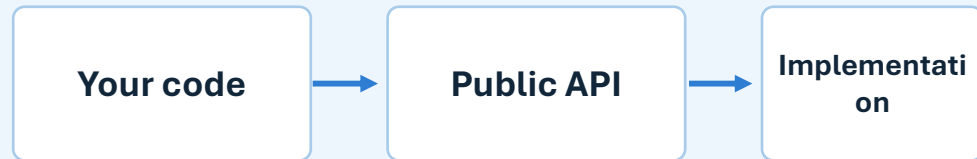
Those benefits become practical only when a language and platform give us useful abstractions, libraries, runtimes, tools, and contracts. Java provides all of these as an ecosystem.

Detailed class syntax starts in Lecture 3. Today we use existing Java types and tools to understand the platform.

APIs and frameworks: where OOP shows up in real software

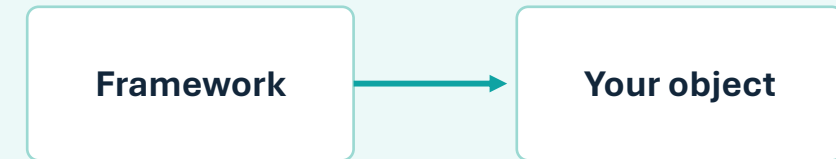
The important idea is the public contract — not the internal implementation.

API: your code calls it



Examples: String, LocalDate, List, Path

Framework: it calls your code



Framework controls the lifecycle and invokes your extension points.

Live example: reuse a Java API instead of reinventing it

[LIVE CODE](#)

7

This demonstrates abstraction and reuse without introducing class design syntax yet.

JAVA

```
import java.time.LocalDate;

public class DeadlineDemo {
    public static void main(String[] args) {
        LocalDate today = LocalDate.now();
        LocalDate quiz = today.plusDays(7);
        LocalDate assignment = today.plusDays(14);

        System.out.println("Today: " + today);
        System.out.println("Quiz: " + quiz);
        System.out.println("Assignment: " + assignment);
    }
}
```

Ask the class

- Did we implement leap years?
- Did we count days in each month?
- Did we handle year boundaries?
- Do we know LocalDate's internal representation?

That is the value of a good public API: meaningful behavior, hidden complexity, reusable implementation.

“Java” can mean several different things

Keep these meanings separate; many beginner confusions come from mixing them.

Java language

syntax, types, classes, statements

Java SE platform

standard APIs + platform contracts

JVM

runtime execution model for class files

JDK

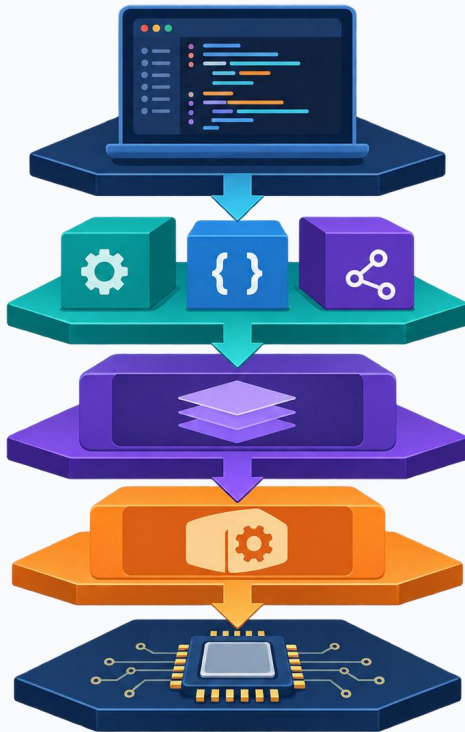
developer tools + compiler + runtime

Java ecosystem

libraries, frameworks, build tools, IDEs

The Java technology stack — a visual model

Your source code sits above reusable APIs and a runtime that ultimately executes on real hardware.



1

Java source code

what you write

2

Java SE APIs

what you reuse

3

JVM

what executes class files

4

Operating system

host services

5

Hardware

CPU + memory

Layer 1 — the Java programming language

The language defines the source-level rules you use to express a program.

SOURCE FILE:
`CourseMessage.java`

```
public class CourseMessage {  
    public static void main(String[] args) {  
        String course = "CSC241";  
        System.out.println("Welcome to " + course);  
    }  
}
```

The language specifies

- syntax and keywords
- types and expressions
- statements and control flow
- classes and interfaces
- compile-time language rules

A .java file is source text. The CPU does not execute it directly.

Layer 2 — Java SE: the standard platform APIs

Java becomes useful quickly because much of the infrastructure is already available as standard APIs.

java.lang

String • Object • Math

java.util

collections • utilities

java.time

date and time

java.io / java.nio

files • streams • paths

java.net

networking

java.sql

database API

Later CSC241 topics — collections, files, serialization, networking, and JDBC — build on these APIs.

Mini demo: one object, several library behaviors

A String object already offers a rich public API — you ask it to perform meaningful operations.

JAVA

```
public class NameDemo {  
    public static void main(String[] args) {  
        String name = "muhammad shahid bhatti";  
  
        System.out.println(name.toUpperCase());  
        System.out.println(name.length());  
        System.out.println(name.contains("shahid"));  
        System.out.println(name.replace("shahid", "S."));  
    }  
}
```

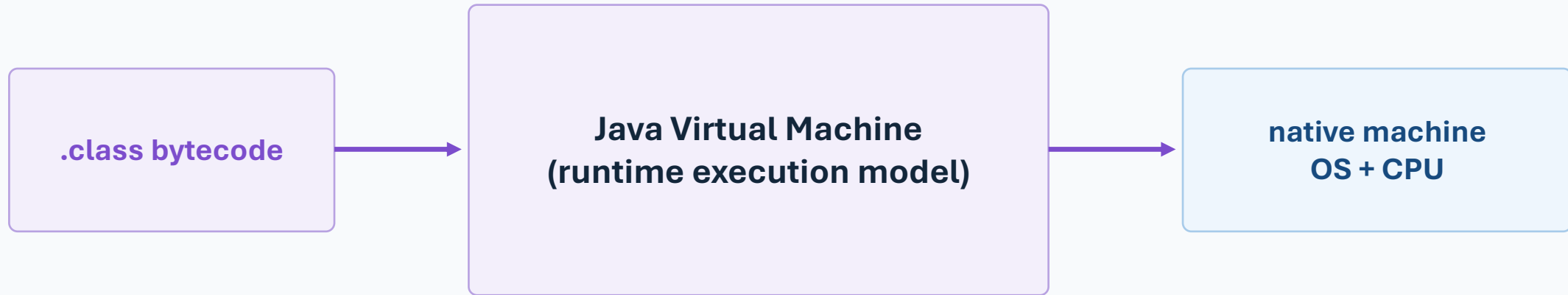
Discussion prompts

- How does toUpperCase() work internally?
- Do we need to know?
- What does the API promise us?
- Why is this easier to reuse than rewriting string logic?

This is OOP in practice: behavior is exposed through a stable type contract.

Layer 3 — JVM: the execution abstraction

The Java Virtual Machine defines how class files are loaded, linked, initialized, and executed.



The JVM is a specified runtime model — not simply “a fake computer.”

Different operating systems use different JVM implementations, but compatible class files follow the same JVM specification.

What happens inside the JVM? — five useful responsibilities

You do not need implementation internals yet; remember the major phases.



Class loading is dynamic: a running application typically loads classes as it needs them.

Reference: Java Virtual Machine Specification, Ch. 5 — Loading, Linking, and Initializing

JRE: an important concept with a modern caveat

Historically, the Java Runtime Environment was the package used to run Java applications.

Conceptual JRE

JVM

+

runtime libraries

+ supporting runtime components

Modern packaging reality

- Developers normally install a JDK.
- The JDK already contains what is needed to compile and run Java.
- Application-specific runtimes can be created with tools such as jlink.
- For exams: know the JRE concept. For practice: do not assume a separate JRE install.

JDK: the developer's toolkit

The Java Development Kit gives you the tools to build, inspect, package, debug, and run Java software.

javac

compiler

java

launcher

javap

class/bytecode inspector

jar

package/archive

jshell

interactive REPL

javadoc

API docs

jdb

debugger

jlink

custom runtime image

An IDE usually calls these tools for you. Understanding them makes setup and debugging much easier.

Your development workflow: editor + JDK + Git

The IDE improves productivity, but the compiler/runtime/toolchain still comes from the JDK.



IDE / editor

write, navigate, refactor, debug

JDK

compile, run, inspect, package

Git

track local changes and history

GitHub

share/publish repository content

Lab 01 connects these pieces. Lecture 2 explains what each piece actually does.

Before coding: verify which Java your machine is using

[SETUP CHECK](#)

A surprising number of “Java problems” are actually PATH or JDK-selection problems.

```
WINDOWS CMD

C:\> java -version
java version "25" ...

C:\> javac -version
javac 25 ...

C:\> where java
C:\Program Files\...\bin\java.exe

C:\> where javac
C:\Program Files\...\bin\javac.exe
```

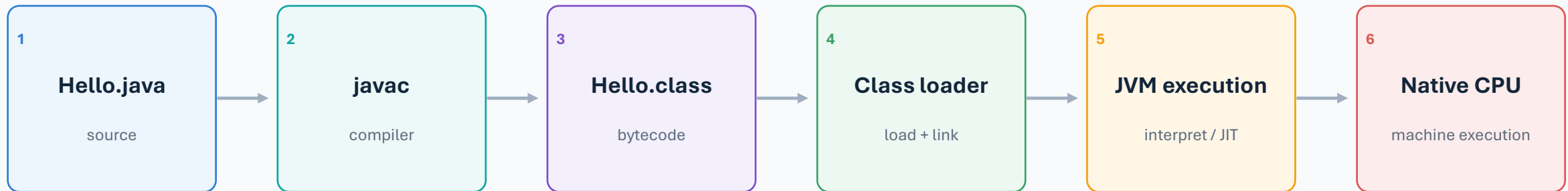
What to check

- java and javac should both be found
- their versions should match the intended JDK
- your IDE project SDK should point to the same JDK
- multiple JDKs are fine — accidental selection is the problem

For this course we use JDK 25 LTS as the semester baseline.

The Java source-to-execution pipeline

This is the central story you should be able to draw from memory.



Compile-time catches source/type errors. Runtime handles loading, dynamic behavior, memory, and actual execution.

Live coding step 1 — create a source file

[LIVE CODE](#)

20

Keep the program intentionally tiny so students can focus on the toolchain.

HelloCSC241.java

```
public class HelloCSC241 {  
    public static void main(String[] args) {  
        System.out.println("Welcome to  
CSC241!");  
    }  
}
```

BEFORE COMPILE

```
C:\CSC241\Lab01> dir  
  
HelloCSC241.java
```

Ask before running anything

“What file should appear after javac succeeds?”

Live coding step 2 — compile, inspect, run

COMPILE

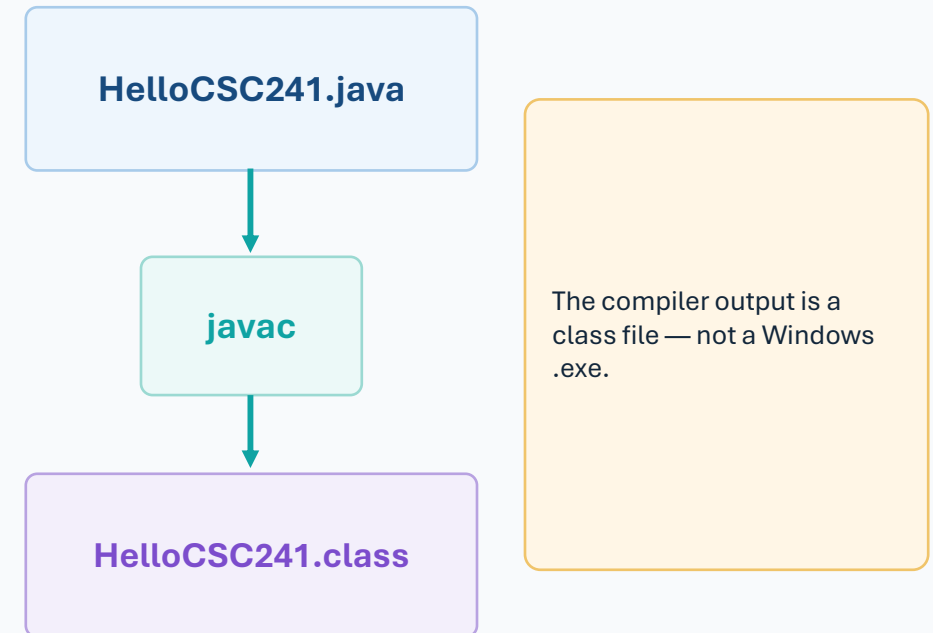
21

The compiler creates a class file; the java launcher then starts the JVM to run the main class.

```
C:\CSC241\Lab01> javac HelloCSC241.java
```

```
C:\CSC241\Lab01> dir
HelloCSC241.java
HelloCSC241.class
```

```
C:\CSC241\Lab01> java HelloCSC241
Welcome to CSC241!
```



What is inside the .class file? — bytecode

A class file is a structured binary format containing JVM instructions and metadata.

CONCEPTUAL `javap -c`
OUTPUT

```
public static void main(java.lang.String[]);
```

Code:

```
0: getstatic      #7  
3: ldc           #13  
5: invokevirtual #15  
8: return
```

Remember

- Bytecode is not Java source.
- Bytecode is not directly tied to one CPU instruction set.
- The JVM maps execution to the host platform.
- `javap` lets us inspect class structure and instructions.

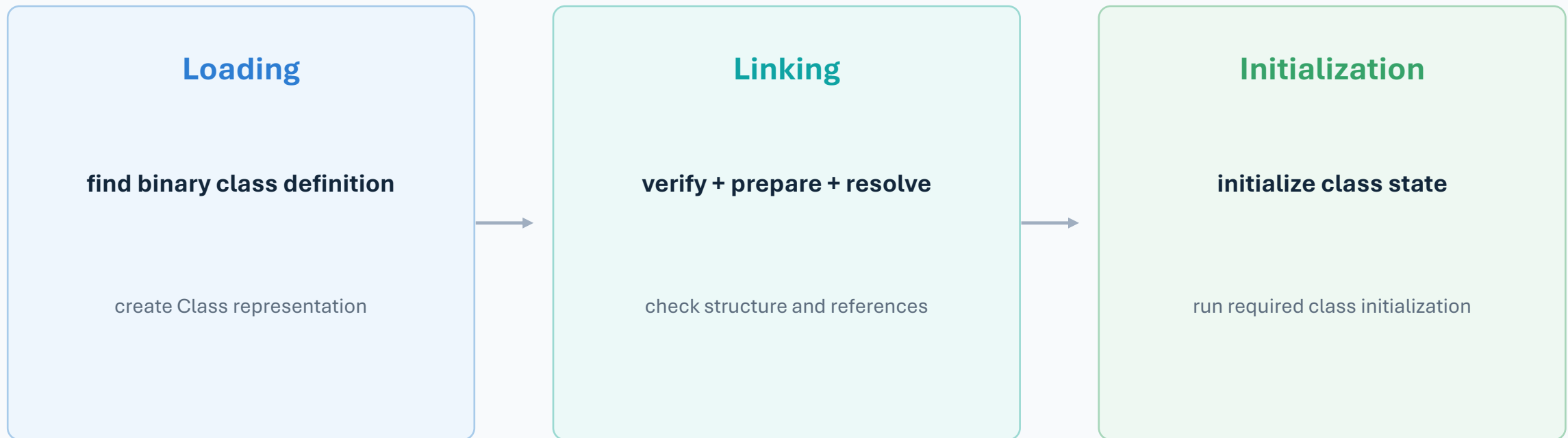


TRY IT

```
C:\> javap -c HelloCSC241
```

Before execution: loading, linking, and initialization

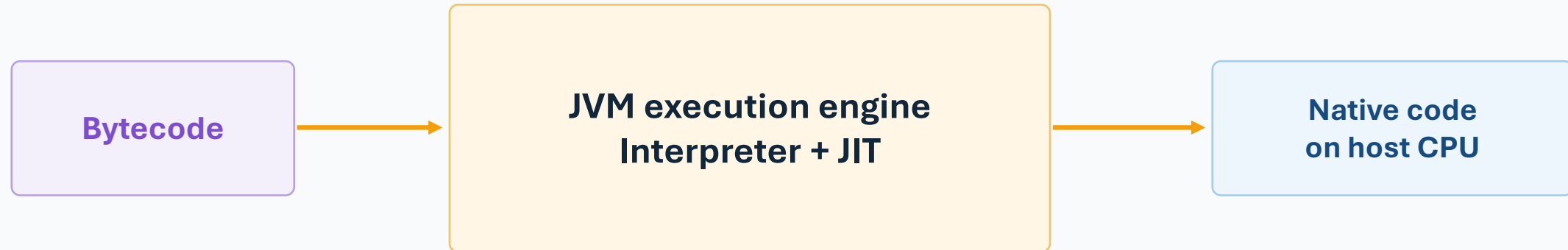
The JVM must obtain and prepare a class definition before it can participate in execution.



Later, constructors, static members, and inheritance will make initialization order much more concrete.

JVM execution: interpreter + JIT compilation

A modern JVM may interpret bytecode initially and compile frequently executed (“hot”) code to native machine code.



Runtime services around execution

garbage collection

threads

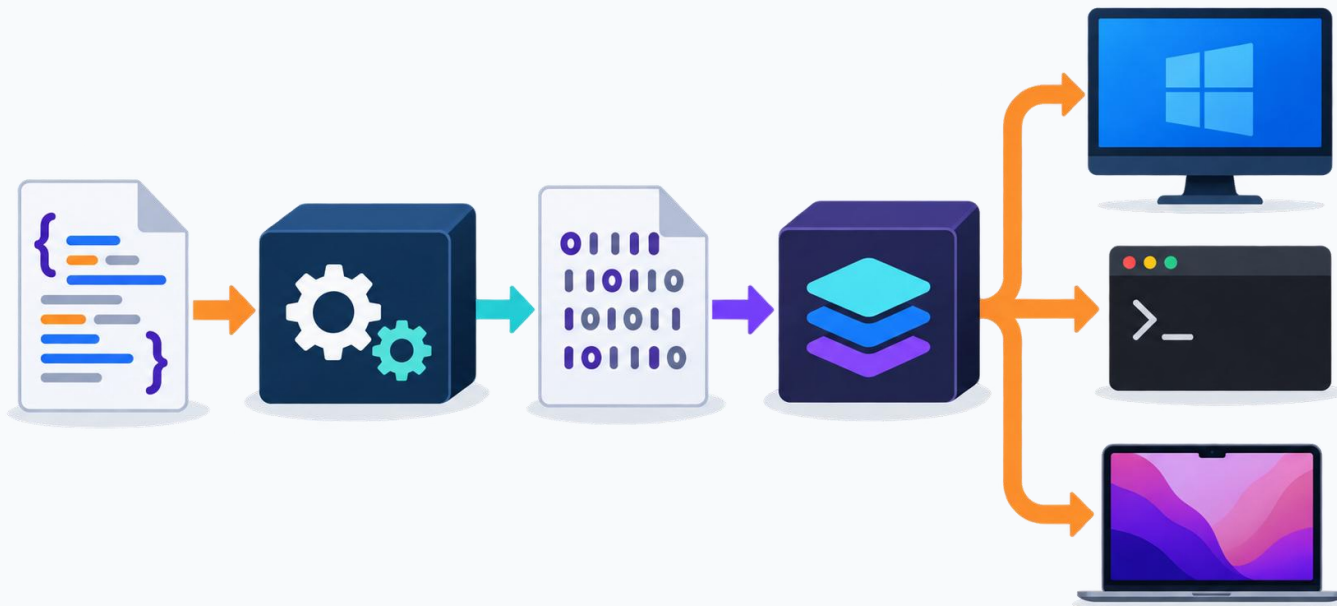
exceptions

runtime checks

Exact optimization details vary by JVM implementation; the core model is bytecode executed by a JVM on the host platform.

Why bytecode helps portability

Compile once to a standardized class-file format; use a platform-specific JVM implementation to execute it.



The practical idea

- same compatible .class file
- different JVM implementation per platform
- same JVM specification / class-file format
- host OS and CPU details handled underneath

Portability still depends on compatible libraries, native dependencies, file paths, and environment assumptions.

Compile-time vs runtime failure — two very different moments

Use this distinction repeatedly throughout the semester.

Compile-time failure

INVALID SOURCE

```
int age = "twenty";
```

...

COMPILER

```
C:\> javac ErrorDemo.java  
error: incompatible types ...
```

The JVM never starts the program.

Runtime failure

VALID SOURCE

```
int a = 10;  
int b = 0;  
System.out.println(a / b);
```

...

RUNTIME

```
C:\> javac ErrorDemo.java  
C:\> java ErrorDemo  
Exception in thread "main" ...
```

Two-minute prediction challenge

THINK

27

Predict before running. Then explain why the API can handle these boundary changes correctly.

PREDICT THE OUTPUT

```
import java.time.LocalDate;

public class DateChallenge {
    public static void main(String[] args) {
        LocalDate d = LocalDate.of(2026, 12, 30);

        System.out.println(d);
        System.out.println(d.plusDays(5));
        System.out.println(d.plusMonths(2));
    }
}
```

Think-pair-share

- What does line 1 print?
- What happens across the year boundary?
- What happens after adding two months?
- What calendar rules did `LocalDate` save us from implementing?

The learning goal is the value of a reusable abstraction — not memorizing `LocalDate` syntax.

JDK release cadence — Fall 2026 classroom snapshot

CURRENT

28

Feature releases arrive every six months; selected releases receive Long-Term Support (LTS).



Course baseline

JDK 25 LTS

Stable semester baseline with long-term support designation.

Important distinction

Newest ≠ best classroom baseline

Sources: Oracle Java SE Support Roadmap; Oracle Java Downloads; OpenJDK JDK 27 project schedule. Checked 12 Sep 2026.

IDE ≠ JDK

The IDE is your productivity layer; the JDK is the Java toolchain underneath.

IDE / editor

- edit and navigate source
- autocomplete and refactor
- run/debug configurations
- project view and integrations
- usually invokes JDK tools for you

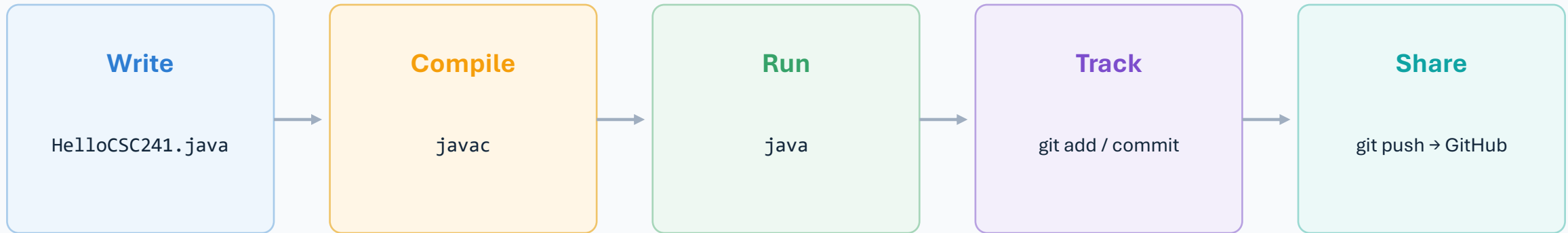
JDK

- javac compiler
- java launcher / runtime
- standard libraries
- inspection and packaging tools
- the SDK your IDE project must point to

When IntelliJ says “Project SDK not defined,” the problem is not your Java source — it is the environment configuration.

Where Git and GitHub fit into the Java workflow

Git does not compile Java. It tracks the evolution of your source and related project files.



Practical rule for the course

Track source files, README files, and useful project assets. Do not treat generated .class files as your primary source of truth.

Quick concept check — fix the misconception

CHECK

31

Ask students to vote: True / False, then justify.

“The JVM is the compiler.”

False — javac compiles; the JVM executes class files.

“A .class file is Windows machine code.”

False — it contains JVM bytecode.

“The IDE includes the Java language.”

False — the IDE is a development environment.

“JDK 26 being newer automatically makes it better for our semester.”

False — classroom stability and LTS support matter.

“If javac fails, the program has already started running.”

False — compilation happens before JVM execution.

Lecture recap & exit ticket

You should now be able to explain the Java platform as a sequence of responsibilities, not just a set of names.

Remember

- Lecture 1 explains why OOP helps organize software.
- Java SE provides reusable platform APIs.
- javac compiles .java source into .class bytecode.
- The JVM loads, links, initializes, and executes class files.
- JDK = developer toolchain; JRE remains an important runtime concept.
- JDK 25 LTS is our semester baseline.

Exit ticket

- Draw the source-to-execution pipeline from memory.
- Differentiate JVM and JDK in one sentence.
- Why is bytecode useful for portability?
- Give one compile-time error and one runtime error.
- Why might we choose JDK 25 LTS instead of JDK 26?

Next lecture: Classes and Objects — anatomy of a class, instantiation, fields, methods, this, identity, state, and references.

Primary references used for the Java platform and current release information.

- 1 Java Language Specification (Java SE 26)
- 2 Java Virtual Machine Specification (Java SE 26), especially Ch. 5
- 3 Oracle javac and java command documentation
- 4 Oracle Java SE Support Roadmap (JDK 25 LTS and future LTS cadence)
- 5 Oracle Java Downloads (JDK 26 current feature release as of 12 Sep 2026)
- 6 OpenJDK JDK 27 project schedule (GA scheduled for 15 Sep 2026)
- 7 Course repository: github.com/mshahidbhatti/CSC241-OOP

This revised deck intentionally minimizes overlap with Lecture 1 and moves the emphasis to Java's modern platform and execution model.

